

DeMaVLA: A Vision-Language-Action Foundation Model for Generalizable Deformable Manipulation

Taiyi Su^{1,*}, Jian Zhu^{1,*,†,‡}, Tianjian Wang^{1,*}, Youzhang He¹, Zitai Huang^{1,2}, Jianjun Zhang^{1,2}, Chong Ma^{1,2}, Hanyang Wang¹, Tianjiao Zhang¹, Munan Yin¹, Weihao Ding¹, Yi Xu^{1,†}

¹AIRC, Midea Group, ²Tongji University

*Equal Contribution, †Corresponding Author, ‡Project Leader

Abstract

Real-world household robots require Vision-Language-Action (VLA) foundation models that can acquire reusable manipulation skills across diverse objects, task conditions, and household environments. Deformable-object folding is a representative challenge, requiring robots to handle clothing items from random initial states across varying categories, geometries, materials, and scenes. However, existing VLA systems commonly train separate policies for different object categories, while naively mixed multi-task training often suffers from task interference and degraded performance. To move beyond category-specific folding policies, we introduce **DeMaVLA**, a VLA foundation model for generalizable **De**formable **Ma**nipulation. DeMaVLA adopts a VLM backbone with an action expert and formulates continuous action generation using flow matching. To improve efficiency, the action expert is constructed by pruning every other transformer layer while preserving layer-wise alignment with the VLM backbone, reducing training and inference cost. DeMaVLA is first pre-trained on approximately 5,000 hours of selected real-world dual-arm demonstrations to acquire general manipulation priors. It is then post-trained on mixed folding data that aggregates self-collected demonstrations and corrective trajectories from real-robot failures across multiple folding tasks through a human-in-the-loop Data Aggregation (DAGger) pipeline. Experiments show that DeMaVLA achieves competitive performance on RoboTwin 2.0 and strong real-world results on our household folding benchmark. These results highlight the value of scalable real-world data, efficient action generation, and corrective learning for general-purpose VLA policies in deformable-object manipulation.

Keywords: Vision-Language-Action Model, Deformable Object Folding, Human-gated DAGger

Date: June 17, 2026

Correspondence: Jian Zhu, Yi Xu

Project Page: <https://demavla.github.io/>



Contents

1	Introduction	3
2	Related Work	3
2.1	VLA Foundation Models	3
2.2	Interactive Imitation Learning	4
3	The DeMaVLA Model	5
3.1	Pre-training Dataset	5
3.2	Model Architecture	5
3.3	Model Training	6
3.4	Human-in-the-Loop DAgger	7
4	Experiments	8
4.1	Comparison on Simulation Benchmark	8
4.2	Comparison on Real-world Benchmark	8
4.3	Pre-training Scaling Analysis	10
5	Conclusion	10

1 Introduction

Vision-Language-Action (VLA) foundation models have become a promising route toward general-purpose robot agents [3, 4, 9, 10, 17, 28, 35, 37]. By grounding visual observations and language instructions into executable actions, recent systems transfer semantic knowledge from large vision-language models [1, 2] into physical control. Yet real-world household deployment requires more than semantic grounding: a robot must reuse manipulation skills under changing object categories, physical states, task conditions, and scene layouts. Deformable-object folding is a representative example of this challenge. Garment items may appear in random, crumpled, or self-occluded states, and folding shirts, pants, towels, or skirts requires distinct action sequences while still relying on shared physical priors such as grasping corners, aligning fabric, smoothing wrinkles, and completing long-horizon bimanual folds.

Existing VLA-based folding systems typically address this diversity through task-specific adaptation, assigning separate policies to different clothing categories. This strategy is difficult to scale in household environments, where the robot must handle diverse foldable items and random initial states without manually selecting and maintaining many specialized models. A straightforward alternative is to train a single policy on mixed folding data, but naive multi-task training can introduce task interference and degrade performance when different object categories require incompatible intermediate states or motion patterns. These limitations motivate a unified folding policy that can share reusable folding priors across clothing categories while preserving task-specific execution.

To move beyond category-specific folding policies, we introduce **DeMaVLA**, a VLA foundation model for generalizable **De**formable **Ma**nipulation. DeMaVLA is designed as a single-checkpoint policy for multi-category bimanual folding on a dual-arm platform. The model adopts a VLM backbone with an action expert and formulates continuous action generation using flow matching [4, 25], enabling smooth action prediction for long-horizon dual-arm manipulation. Specifically, our action expert is built from the LLM component of Qwen3-VL, enabling it to share the same transformer architecture as the language backbone. However, directly using the full LLM as an action expert introduces substantial computation, especially because flow-based action generation requires repeated expert forward passes. To retain the aligned architecture while improving efficiency, we prune every other transformer layer in the action expert, reducing training and inference cost without changing the overall VLA interface.

DeMaVLA is trained with both scalable pre-training and failure-driven post-training. It is first pre-trained on approximately 5,000 hours of selected real-world dual-arm demonstrations from public and self-collected datasets to acquire general manipulation priors. It is then post-trained on mixed folding data that combines self-collected demonstrations with corrective trajectories collected through a human-in-the-loop Data Aggregation (Dagger) pipeline. By rolling out the mixed folding policy on real robots and correcting its failures across different clothing categories, this pipeline directly targets the failure modes of the unified model and substantially improves its multi-category folding capability. We evaluate DeMaVLA on RoboTwin 2.0 [11] and a real-world household folding benchmark covering multiple clothing categories and random initial states.

Our main contributions are threefold. First, we highlight the importance of implementation-level design in building effective VLA policies, including an LLM-based action expert, skip-layer pruning, flow-matching action generation, and training-time RTC. Second, we demonstrate that scalable real-world pre-training is critical for deformable-object manipulation, as DeMaVLA benefits from approximately 5,000 hours of dual-arm demonstrations and mixed folding data. Third, DeMaVLA achieves strong performance in both simulation and real-world household folding, validating the effectiveness of combining large-scale data, efficient model design, and failure-driven corrective learning.

2 Related Work

2.1 VLA Foundation Models

Vision-Language-Action (VLA) models have changed robot policy learning by connecting visual perception, language instructions, and action generation in a unified model. Compared with conventional task-specific policies, VLAs provide a scalable interface for instruction following and multi-task manipulation, and have

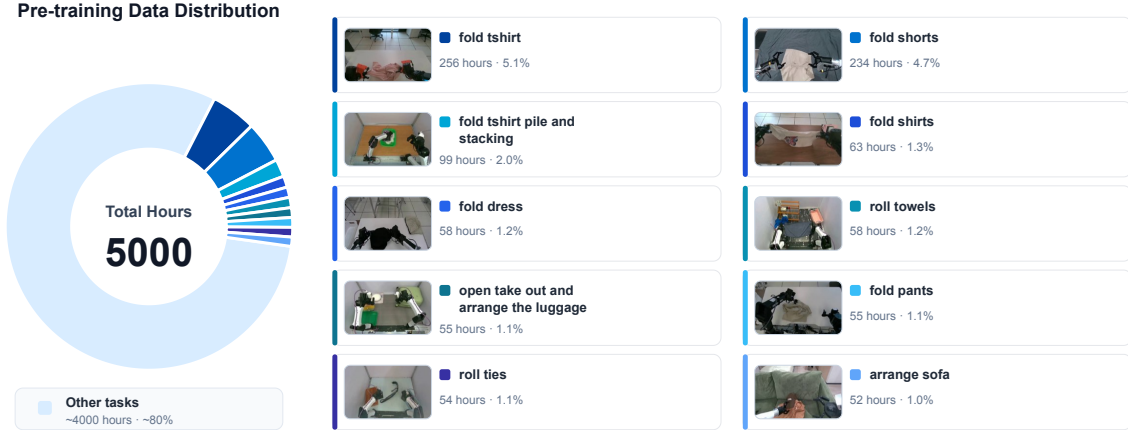


Figure 1 Distribution of the DeMaVLA pre-training dataset.

shown strong potential in transferring semantic knowledge into robot control [3, 9, 10, 16, 28, 35, 37, 38]. Most recent VLAs build on pre-trained vision-language models (VLMs), whose image-text pre-training provides useful priors for perception, grounding, and instruction understanding [1, 2]. Robot actions are then learned by adding policy heads or action experts on top of the VLM context [4, 20].

Scaling embodied data is equally important. Recent foundation policies collect robot trajectories across diverse embodiments, tasks, and environments to improve generalization [18, 26, 32, 33]. Related systems further explore cross-embodiment learning, open-world robot datasets, real-time execution, and generalist manipulation policies [7, 9, 12, 24, 30, 36, 37]. For continuous control, flow matching has become a practical formulation because it models multimodal action distributions while generating smooth action chunks [4, 25].

DeMaVLA follows this VLM- and flow-based VLA paradigm, but focuses on a different deployment goal: learning a unified folding policy across multiple clothing categories and random initial states. Instead of treating each category as an independent downstream adaptation, DeMaVLA learns shared folding priors from large-scale dual-arm data and applies them through a single checkpoint for diverse folding skills.

2.2 Interactive Imitation Learning

Human demonstrations provide direct supervision for complex robot manipulation, making imitation learning a central training strategy for modern VLA policies. In the standard passive setting, expert trajectories are collected first and a policy is trained offline through behavior cloning. This approach is simple and stable, but it suffers from covariate shift: once the learned policy deviates from the demonstration distribution, errors can compound and the robot may fail to recover. Interactive imitation learning addresses this limitation by collecting supervision from the learner’s own state distribution [8].

Data Aggregation (DAgger) formalizes this idea by iteratively executing the current policy, querying expert actions on visited states, and aggregating the newly labeled data into the training set [29]. Compared with passive behavior cloning, DAgger-style methods improve sample efficiency and robustness because they target states where the learned policy is likely to make mistakes. Human-in-the-loop variants make this idea practical for real robots by allowing a human expert to decide when to intervene. Prior work studies budget-aware querying, teleoperated corrections, recovery data, data augmentation, and human-AI copilot systems [14, 15, 21–23, 27]. Human-Gated DAgger further lets the operator take over only when needed and release control after correction [19, 31, 34].

DeMaVLA adopts this human-gated formulation for multi-category bimanual folding. After initializing the policy from the pre-trained checkpoint, we collect DAgger data by rolling out the policy on real robots and recording corrective human interventions at failure states during mixed folding tasks. This is important for long-horizon deformable-object manipulation, where small errors in grasping, alignment, or intermediate fabric states can compound into failure modes that are rare in static demonstration datasets. By aggregating

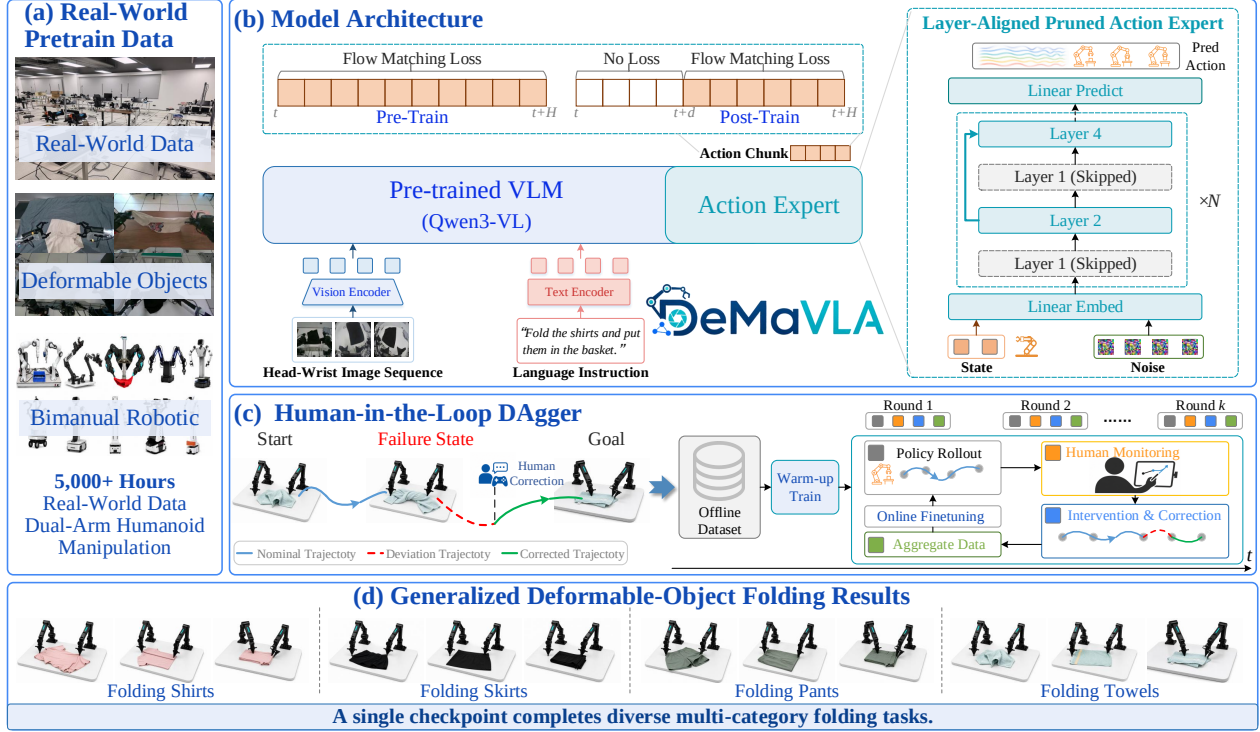


Figure 2 Overview of DeMaVLA. DeMaVLA is trained from real-world dual-arm demonstrations and integrates a Qwen3-VL backbone, a layer-aligned pruned action expert, flow-matching action generation, training-time RTC for asynchronous execution, and human-in-the-loop DAGger for corrective real-world learning. A single checkpoint is used to perform diverse multi-category folding tasks.

these corrections into post-training, DeMaVLA directly targets the failure modes of the unified folding policy and improves its robustness across different clothing categories.

3 The DeMaVLA Model

3.1 Pre-training Dataset

Our pre-training dataset contains approximately 5,000 hours of real-world demonstrations from dual-arm humanoid manipulation. Fig. 1 visualizes the data distribution. The pre-training dataset covers a wide range of real-world bimanual manipulation skills, including household organization, folding, object transfer, industrial handling, and long-horizon coordinated interaction. This broad coverage allows DeMaVLA to acquire general manipulation priors such as reaching, grasping, placement, object rearrangement, and two-arm coordination. At the same time, the folding-oriented subsets provide dense supervision for garment handling, spreading, alignment, and deformable-object manipulation, which are important for the downstream household folding tasks. The combination of broad manipulation data and task-relevant folding data helps DeMaVLA benefit from general-purpose pre-training while remaining compatible with the target ALOHA-style [13] dual-arm deployment setting.

3.2 Model Architecture

DeMaVLA follows a VLM-conditioned action-generation architecture for real-world bimanual manipulation. At each timestep t , the model receives multi-view image observations, a language instruction, the robot proprioceptive state, and a noisy action chunk. It predicts the continuous action vector field used for flow-based action generation. The VLM backbone provides visual-language context, while the action expert processes robot-specific inputs and produces continuous bimanual action predictions.

Table 1 Parameter count of DeMaVLA.

Module Name	Total Params
Vision Encoder	0.4B
Base Language Model	4.0B
Action Expert	2.2B
Total Model	6.6B

Formally, the observation context is defined as

$$\mathbf{O}_t = [\mathbf{I}_t^1, \mathbf{I}_t^2, \mathbf{I}_t^3, \ell_t, \mathbf{q}_t], \quad (1)$$

where $\mathbf{I}_t^{1,2,3}$ denote the three operational views, ℓ_t is the natural language instruction, and $\mathbf{q}_t$ is the robot proprioceptive state. The corresponding action chunk is defined as

$$\mathbf{A}_t = [\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}], \quad (2)$$

where H is the action horizon and each $\mathbf{a}_t$ is a continuous dual-arm control command.

DeMaVLA adopts Qwen3-VL [1] as the VLM backbone. The multi-view image observations and the language instruction are tokenized into visual-language tokens, which provide semantic and spatial context for action generation. Since standard VLM pre-training does not include robot proprioception or continuous action prediction, DeMaVLA introduces an action expert to handle robot-specific tokens. The proprioceptive state $\mathbf{q}_t$ is projected into the transformer embedding space through a linear layer. The noisy action chunk $\mathbf{A}_{t,s}$ is represented as H action tokens, one for each future control step. These tokens are processed by the action expert while being conditioned on the VLM context, and the action expert outputs the vector field used to denoise the action chunk.

Layer-Aligned Pruned Action Expert. To align the action expert with the VLM backbone, we build it by reusing the LLM transformer structure of Qwen3-VL. This design preserves a layer-wise correspondence between the visual-language backbone and the action branch, making it straightforward for robot-specific tokens to condition on hierarchical VLM representations. However, directly using the full LLM structure as an action expert is inefficient for flow-based action generation, since the expert must be evaluated repeatedly during denoising.

To retain the aligned architecture while improving efficiency, DeMaVLA constructs a layer-aligned pruned action expert. Let the VLM backbone contain L transformer layers. A full action expert would mirror these L layers for layer-wise conditioning. Instead, DeMaVLA keeps a subset of expert layers and removes the remaining layers from the action branch. For two consecutive retained expert layers, the output of the earlier retained layer is directly passed to the next retained layer. The retained expert layers still correspond to selected layers of the VLM backbone, preserving hierarchical visual-language conditioning while reducing the number of action-expert transformer blocks by approximately half. This produces a lightweight action branch without changing the input-output formulation of the VLA policy.

The resulting architecture combines a Qwen3-VL backbone, a layer-aligned pruned action expert, and flow-matching-based continuous action generation. The parameter counts of the main components are summarized in Table 1.

3.3 Model Training

Pre-training. During pre-training, DeMaVLA learns a general bimanual manipulation policy from the trajectory mixture described in Sec. 3.1. Given the observation context $\mathbf{O}_t$, which contains multi-view images, a language instruction, and robot proprioception, the model predicts a future action chunks. Instead of discretizing robot actions into language tokens, we follow prior VLA policies [4, 25] and model continuous action chunks with conditional flow matching, which is well suited for smooth dual-arm control.

Concretely, the action expert is trained to denoise noisy action chunks conditioned on the VLM context and robot state. This objective encourages the model to learn the conditional action distribution $p(\mathbf{A}_t \mid \mathbf{O}_t)$ while preserving the continuity of the action space. During this stage, the VLM provides visual-language conditioning, and the action expert learns to map noisy action tokens and proprioceptive states into executable dual-arm trajectories. The resulting pre-trained policy provides DeMaVLA with broad manipulation priors before downstream adaptation to real-time folding tasks.

Post-training. For real-world deployment, DeMaVLA is post-trained with training-time real-time chunking (RTC) [5]. Standard action-chunking policies may suffer from inference latency: while the model is generating the next action chunk, the robot must continue executing actions from the previously predicted chunk. Instead of applying inference-time inpainting or guidance [6], training-time RTC simulates this latency during post-training and teaches the model to generate future actions conditioned on already committed actions.

Specifically, each action chunk is divided into a committed prefix and a predicted postfix. The committed prefix corresponds to actions that will already be executed before the new prediction becomes available, while the postfix contains the future actions that the model must generate. During post-training, the prefix is kept clean as conditioning context, and flow noise is applied only to the postfix. The training loss is computed only on the postfix actions, encouraging DeMaVLA to produce temporally consistent future trajectories while respecting the committed prefix. At inference time, DeMaVLA conditions on the previously committed actions and predicts the remaining future actions, enabling asynchronous action-chunk execution without additional inpainting or backpropagation-based guidance.

3.4 Human-in-the-Loop Dagger

Although large-scale pre-training provides DeMaVLA with broad manipulation priors, real-world multi-category folding still exposes the policy to failure states that are difficult to cover with passive demonstrations alone. Small errors in grasping, cloth alignment, or intermediate folding poses can accumulate over long horizons and lead the robot into states outside the demonstration distribution. To improve robustness under such failures, we adopt a human-gated interactive imitation learning pipeline inspired by HG-Dagger [19].

We initialize the policy with behavior cloning on the pre-training and task demonstration data. During interactive teaching, the mixed folding policy is deployed on the real robot and executes folding rollouts across different clothing categories. A human operator monitors the rollout and intervenes when the policy enters an unsafe, unstable, or clearly suboptimal state. Once intervention is triggered, control is fully transferred to the human expert, who corrects the behavior and guides the robot back to a valid task state. After the correction is completed, control is released back to the autonomous policy. This human-gated mechanism avoids requiring the expert to continuously label states under mixed control, and instead collects high-value corrective data around the unified policy’s actual failure modes.

Formally, let π_θ denote the current DeMaVLA policy and π_H denote the human expert. At timestep t , the human operator provides a binary gate $g_t \in \{0, 1\}$, where $g_t = 1$ indicates human intervention. The executed policy is

$$\pi_{\text{exec}}(\mathbf{o}_t) = g_t \pi_H(\mathbf{o}_t) + (1 - g_t) \pi_\theta(\mathbf{o}_t), \quad (3)$$

where $\mathbf{o}_t$ denotes the robot observation. In practice, $\mathbf{o}_t$ contains multi-view images, language instruction, and proprioceptive state. We record expert-labeled correction data only during intervention intervals:

$$\mathcal{D}_{\text{int}} = \{(\mathbf{o}_t, \mathbf{a}_t^H) \mid g_t = 1\}, \quad (4)$$

where $\mathbf{a}_t^H = \pi_H(\mathbf{o}_t)$ is the action provided by the human operator.

After each round of interactive data collection, the intervention data are aggregated into the training set:

$$\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_{\text{int}}. \quad (5)$$

We then continue post-training DeMaVLA on the updated dataset using the same flow-matching objective. For deployment, we train the final policy on the aggregated dataset collected across all interactive rounds.

Table 2 Evaluation on RoboTwin 2.0 simulation benchmark under clean and randomized settings.

π_0 [4]		$\pi_{0.5}$ [17]		X-VLA [37]		ABot-M0 [33]		LingBot-VLA [32]		DeMaVLA	
Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.
65.92	58.40	82.74	76.76	72.80	72.84	80.42	81.16	86.50	85.34	88.42	86.78

Compared with passive imitation learning, this procedure focuses supervision on states where the mixed folding policy is likely to fail. For multi-category folding tasks, such corrective demonstrations are especially valuable because they teach recovery behaviors for misaligned cloth, incomplete grasps, and partially folded configurations. By aggregating these corrections into post-training, DeMaVLA directly targets the failure modes of the unified folding policy and improves its robustness across different clothing categories. As the policy improves, the frequency of human intervention naturally decreases, making the intervention rate a practical signal for measuring whether additional interactive data collection is still needed.

4 Experiments

4.1 Comparison on Simulation Benchmark

To comprehensively evaluate DeMaVLA, we conduct experiments on the RoboTwin 2.0 simulation benchmark [11], which contains 50 bimanual manipulation tasks under both clean and randomized settings. The clean setting uses fixed initial configurations, while the randomized setting varies object poses and scene layouts. We compare DeMaVLA against several representative VLA foundation models including: π_0 [4], $\pi_{0.5}$ [17], X-VLA [37], ABot-M0 [33] and LingBot-VLA [32]. All methods are evaluated on the same RoboTwin task suite under clean and randomized settings, and we report the average success rate across the 50 simulation tasks. Table 2 summarizes the average performance across all tasks, and the full per-task results are reported in Appendix Table A.1.

As shown in Table 2, DeMaVLA achieves the best average performance under both settings, reaching 88.42% in the clean setting and 86.78% in the randomized setting. The gain under randomized scenes suggests that DeMaVLA preserves strong robustness when object poses and scene layouts vary. Overall, these results indicate that the proposed pre-training recipe and efficient action-generation design provide competitive generalization on diverse bimanual manipulation tasks.

4.2 Comparison on Real-world Benchmark

Task Setup. We further evaluate DeMaVLA on a real-world household folding benchmark using an ALOHA-style dual-arm robot. The benchmark contains four representative folding tasks: folding a shirt, folding a skirt, folding pants, and folding a towel. These tasks cover different deformable object geometries, aspect ratios, material properties, and folding routines, making them suitable for evaluating whether a VLA policy can generalize beyond a single object category. Each trial evaluates the complete household folding procedure rather than only the final table-top folding stage. The target item is first randomly dropped into a basket placed beside the table, which differs from settings where the object is already laid flat on the table or neatly stacked in the basket. The robot must pick up the item from the basket, place it on the table, unfold and spread it into a foldable configuration, and then complete the target folding routine. Each task is evaluated over 20 real-world trials using two sets of different items.

Evaluation Protocol. A trial is considered successful if the robot completes the full procedure and produces a final folded configuration that satisfies the task-specific completion criteria. A trial is counted as a failure if either (1) the robot does not finish the task within 5 minutes, or (2) the garment falls off the table during execution. We report two metrics for each task: Success Rate (SR), defined as the proportion of successful trials within the 5-minute time limit, and Completion Time, defined as the average task duration. Higher SR indicates better task reliability, while lower completion time indicates more efficient execution. Failed trials are counted as 5 minutes when computing average completion time, so the time metric reflects both

Table 3 Single-task comparison on the shirt folding task.

Training Setting	SR	Time
Single-task π_0	80.0%	2:13
Single-task DeMaVLA	100.0%	2:04

Table 4 Real-world evaluation on the household folding benchmark.

Method	Shirt		Skirt		Pant		Towel		Average	
	SR	Time	SR	Time	SR	Time	SR	Time	SR	Time
π_0	90.0%	1:55	95.0%	1:03	65.0%	3:01	55.0%	3:44	76.3%	2:26
DeMaVLA	95.0%	2:15	100.0%	1:30	75.0%	3:01	100.0%	2:26	92.5%	2:18

execution speed and failure frequency. All methods are evaluated under the same robot hardware, controller frequency, and initialization protocol.

Compared Methods. We compare DeMaVLA with a state-of-the-art VLA baseline. Both methods use training-time RTC for asynchronous chunk execution:

- π_0 . A strong VLA baseline adapted from the released π_0 base model. We fine-tune it on our folding tasks with the same training-time RTC setting.
- DeMaVLA. Our proposed model, trained with the DeMaVLA architecture, large-scale real-world pre-training data, human-in-the-loop DAgger, and training-time RTC.

Single-task Comparison. We first compare single-task policies on the shirt folding task, where each model is trained and evaluated only on shirt demonstrations. This setting tests whether the proposed architecture and training recipe provide stronger task-specific folding capability when the task distribution is fixed. As shown in Table 3, single-task DeMaVLA achieves a 100.0% SR, improving over the single-task π_0 baseline by 20.0 percentage points. DeMaVLA also slightly reduces the average completion time from 2:13 to 2:04.

Multi-task Comparison. We then evaluate whether one checkpoint can solve multiple folding tasks. In this setting, both π_0 and DeMaVLA are trained on mixed demonstrations from shirt, skirt, pant, and towel folding, and are evaluated on all four tasks. The mixed folding dataset contains 37 hours of real-robot data in total, including 21.9 hours for folding shirts, 8.4 hours for folding skirts, 3.7 hours for folding pants, and 3.0 hours for folding towels. The DAgger data are collected incrementally following this task order, so later tasks require less additional data as the policy gradually improves and transfers folding priors from earlier tasks.

Table 4 reports success rate and average completion time for each task. DeMaVLA achieves a higher average SR than π_0 across the four tasks, improving from 76.3% to 92.5%. The largest gain appears on towel folding, where DeMaVLA reaches 100.0% SR compared with 55.0% for π_0 , showing stronger robustness on a highly deformable and visually ambiguous object category. DeMaVLA also improves shirt, skirt, and pant folding SR, while maintaining a lower average completion time across all tasks (2:18 vs. 2:26). These results indicate that DeMaVLA can share folding priors across garment categories and execute them through a single multi-task policy.

Discussion. The real-world benchmark is designed to evaluate two capabilities. First, the comparison with π_0 tests whether DeMaVLA provides stronger real-world folding performance than a general VLA baseline under the same RTC deployment setting. Second, the single-task and multi-task results examine whether DeMaVLA can use one checkpoint to solve multiple folding tasks while retaining strong performance on the shirt task. Together, these evaluations show that DeMaVLA behaves as a unified fold-anything policy rather than a collection of independently fine-tuned task policies.

Table 5 Pre-training scaling analysis on the real-world shirt-folding task. All checkpoints are post-trained with the same shirt-folding data.

Pre-training Data	SR	Time
500 hours	55.0%	3:34
2,500 hours	70.0%	3:21
5,000 hours	100.0%	2:04

4.3 Pre-training Scaling Analysis

We further study how the scale of pre-training data affects downstream real-world folding performance. To isolate the effect of pre-training scale, we compare three DeMaVLA checkpoints trained with different amounts of selected real-world pre-training data: 500 hours, 2,500 hours, and 5,000 hours. All checkpoints are then post-trained with the same shirt-folding data and evaluated under the same real-world protocol.

The results in Table 5 show a clear scaling trend. As the pre-training data increases from 500 to 5,000 hours, the SR improves from 55.0% to 100.0%, while the average completion time decreases from 3:34 to 2:04. The intermediate 2,500-hour checkpoint also shows consistent improvement, achieving a 70.0% SR and 3:21 average completion time. These results indicate that scaling real-world pre-training data improves not only downstream task success but also execution efficiency after post-training.

5 Conclusion

We introduced DeMaVLA, a VLA foundation model for generalizable deformable manipulation. DeMaVLA moves beyond category-specific folding policies by using a single checkpoint to handle multiple household folding tasks with different garments, initial states, and long-horizon bimanual routines. Its design combines a Qwen3-VL backbone, an LLM-based layer-aligned action expert, skip-layer pruning, flow-matching action generation, and training-time RTC, showing that implementation-level architecture and deployment choices are central to effective real-world VLA policies. To support scalable learning, DeMaVLA is pre-trained on approximately 5,000 hours of selected real-world dual-arm demonstrations and then post-trained with mixed folding data and human-gated DAgger corrections that target policy failure states. Experiments on RoboTwin 2.0 and our real-world household folding benchmark validate this combination of large-scale data, efficient model design, and failure-driven corrective learning, demonstrating strong performance in both simulation and real-world multi-category folding.

References

- [1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [2] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, et al. Paligemma: A versatile 3b vlm for transfer. *arXiv preprint arXiv:2407.07726*, 2024.
- [3] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [5] Kevin Black, Allen Z Ren, Michael Equi, and Sergey Levine. Training-time action conditioning for efficient real-time chunking. *arXiv preprint arXiv:2512.05964*, 2025.
- [6] Kevin Black, Manuel Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. *Advances in Neural Information Processing Systems*, 38:33383–33407, 2026.
- [7] Rui Cai, Jun Guo, Xinze He, Piaopiao Jin, Jie Li, Bingxuan Lin, Futeng Liu, Wei Liu, Fei Ma, Kun Ma, et al. Xiaomi-robotics-0: An open-sourced vision-language-action model with real-time execution. *arXiv preprint arXiv:2602.12684*, 2026.
- [8] Carlos Celemin, Rodrigo Pérez-Dattari, Eugenio Chisari, Giovanni Franzese, Leandro de Souza Rosa, Ravi Prakash, Zlatan Ajanović, Marta Ferraz, Abhinav Valada, and Jens Kober. Interactive imitation learning in robotics: A survey. *Foundations and Trends® in Robotics*, 10(1-2):1–197, 2022.
- [9] Jun Cen, Siteng Huang, Yuqian Yuan, Kehan Li, Hangjie Yuan, Chaohui Yu, Yuming Jiang, Jiayan Guo, Xin Li, Hao Luo, et al. Rynnvla-002: A unified vision-language-action and world model. *arXiv preprint arXiv:2511.17502*, 2025.
- [10] Chilam Cheang, Sijin Chen, Zhongren Cui, Yingdong Hu, Liqun Huang, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Xiao Ma, et al. Gr-3 technical report. *arXiv preprint arXiv:2507.15493*, 2025.
- [11] Tianxing Chen, Zanzin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.
- [12] StarVLA Community. Starvla: A lego-like codebase for vision-language-action model developing. *arXiv preprint arXiv:2604.05014*, 2026.
- [13] Zipeng Fu, Tony Z Zhao, and Chelsea Finn. Mobile aloha: Learning bimanual mobile manipulation with low-cost whole-body teleoperation. *arXiv preprint arXiv:2401.02117*, 2024.
- [14] Ryan Hoque, Ashwin Balakrishna, Ellen Novoseller, Albert Wilcox, Daniel S Brown, and Ken Goldberg. Thriftydagger: Budget-aware novelty and risk gating for interactive imitation learning. *arXiv preprint arXiv:2109.08273*, 2021.
- [15] Zheyuan Hu, Robyn Wu, Naveen Enock, Jasmine Li, Riya Kadakia, Zackory Erickson, and Aviral Kumar. Rac: Robot learning for long-horizon tasks by scaling recovery and correction. *arXiv preprint arXiv:2509.07953*, 2025.
- [16] Physical Intelligence, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, et al. $\pi_{0.6}^*$: a vla that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.
- [17] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [18] Tao Jiang, Tianyuan Yuan, Yicheng Liu, Chenhao Lu, Jianning Cui, Xiao Liu, Shuiqi Cheng, Jiyang Gao, Huazhe Xu, and Hang Zhao. Galaxea open-world dataset and g0 dual-system vla model. *arXiv preprint arXiv:2509.00576*, 2025.

- [19] Michael Kelly, Chelsea Sidrane, Katherine Driggs-Campbell, and Mykel J Kochenderfer. Hg-dagger: Interactive imitation learning with human experts. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 8077–8083. IEEE, 2019.
- [20] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [21] Misha Laskin, Kimin Lee, Adam Stooke, Lerrel Pinto, Pieter Abbeel, and Aravind Srinivas. Reinforcement learning with augmented data. *Advances in neural information processing systems*, 33:19884–19895, 2020.
- [22] Quanyi Li, Zhenghao Peng, and Bolei Zhou. Efficient learning of safe driving policy via human-ai copilot optimization. *arXiv preprint arXiv:2202.10341*, 2022.
- [23] Yunfei Li, Xiao Ma, Jiafeng Xu, Yu Cui, Zhongren Cui, Zhigang Han, Liquan Huang, Tao Kong, Yuxiao Liu, Hao Niu, et al. Gr-rl: Going dexterous and precise for long-horizon robotic manipulation. *arXiv preprint arXiv:2512.01801*, 2025.
- [24] Xuewu Lin, Tianwei Lin, Yun Du, Hongyu Xie, Yiwei Jin, Jiawei Li, Shijie Wu, Qingze Wang, Mengdi Li, Mengao Zhao, et al. Holobrain-0 technical report. *arXiv preprint arXiv:2602.12062*, 2026.
- [25] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- [26] Hao Luo, Ye Wang, Wanpeng Zhang, Sipeng Zheng, Ziheng Xi, Chaoyi Xu, Haiweng Xu, Haoqi Yuan, Chi Zhang, Yiqing Wang, et al. Being-h0. 5: Scaling human-centric robot learning for cross-embodiment generalization. *arXiv preprint arXiv:2601.12993*, 2026.
- [27] Ajay Mandlekar, Danfei Xu, Roberto Martín-Martín, Yuke Zhu, Li Fei-Fei, and Silvio Savarese. Human-in-the-loop imitation learning using remote teleoperation. *arXiv preprint arXiv:2012.06733*, 2020.
- [28] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- [29] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635, 2011.
- [30] GigaBrain Team, Boyuan Wang, Bohan Li, Chaojun Ni, Guan Huang, Guosheng Zhao, Hao Li, Jie Li, Jindi Lv, Jingyu Liu, et al. Gigabrain-0.5 m*: a vla that learns from world model-based reinforcement learning. *arXiv preprint arXiv:2602.12099*, 2026.
- [31] Philipp Wu, Yide Shentu, Qiayuan Liao, Ding Jin, Menglong Guo, Koushil Sreenath, Xingyu Lin, and Pieter Abbeel. Robocopilot: Human-in-the-loop interactive imitation learning for robot manipulation. *arXiv preprint arXiv:2503.07771*, 2025.
- [32] Wei Wu, Fan Lu, Yunnan Wang, Shuai Yang, Shi Liu, Fangjing Wang, Qian Zhu, He Sun, Yong Wang, Shuailei Ma, et al. A pragmatic vla foundation model. *arXiv preprint arXiv:2601.18692*, 2026.
- [33] Yandan Yang, Shuang Zeng, Tong Lin, Xinyuan Chang, Dekang Qi, Junjin Xiao, Haoyun Liu, Ronghan Chen, Yuzhi Chen, Dongjie Huo, et al. Abot-m0: Vla foundation model for robotic manipulation with action manifold learning. *arXiv preprint arXiv:2602.11236*, 2026.
- [34] Checheng Yu, Chonghao Sima, Gangcheng Jiang, Hai Zhang, Haoguang Mai, Hongyang Li, Huijie Wang, Jin Chen, Kaiyang Wu, Li Chen, et al. χ_0 : Resource-aware robust manipulation via taming distributional inconsistencies. *arXiv preprint arXiv:2602.09021*, 2026.
- [35] Andy Zhai, Brae Liu, Bruno Fang, Chalse Cai, Ellie Ma, Ethan Yin, Hao Wang, Hugo Zhou, James Wang, Lights Shi, et al. Igniting vlms toward the embodied space. *arXiv preprint arXiv:2509.11766*, 2025.
- [36] Tianle Zhang, Zhihao Yuan, Dafeng Chi, Peidong Liu, Dongwei Li, Kejun Hu, Likui Zhang, Junnan Nie, Ziming Wei, Zengjue Chen, et al. Joyai-ra 0.1: A foundation model for robotic autonomy. *arXiv preprint arXiv:2604.20100*, 2026.

- [37] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, et al. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*, 2025.
- [38] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.

Appendix

Table A.1 Evaluation on RoboTwin 2.0 Simulation Benchmark.

Simulation Task	π_0		$\pi_{0.5}$		X-VLA		LingBot-VLA		DeMaVLA	
	Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.	Clean	Rand.
<i>Adjust Bottle</i>	99%	95%	100%	99%	100%	99%	100%	100%	99%	100%
<i>Beat Block Hammer</i>	79%	84%	96%	93%	92%	88%	87%	91%	79%	85%
<i>Blocks Ranking RGB</i>	80%	63%	92%	85%	83%	83%	92%	91%	95%	95%
<i>Blocks Ranking Size</i>	14%	5%	49%	26%	67%	74%	66%	73%	72%	68%
<i>Click Alarmclock</i>	77%	68%	98%	89%	99%	99%	93%	26%	98%	100%
<i>Click Bell</i>	71%	48%	99%	66%	100%	100%	32%	19%	96%	98%
<i>Dump Bin Bigbin</i>	88%	83%	92%	97%	79%	77%	97%	92%	91%	94%
<i>Grab Roller</i>	98%	94%	100%	100%	100%	100%	100%	99%	100%	100%
<i>Handover Block</i>	47%	31%	66%	57%	73%	37%	80%	83%	93%	83%
<i>Handover Mic</i>	97%	97%	98%	97%	0%	0%	94%	98%	95%	96%
<i>Hanging Mug</i>	14%	11%	18%	17%	23%	27%	32%	27%	46%	35%
<i>Lift Pot</i>	80%	72%	96%	85%	99%	100%	100%	99%	97%	94%
<i>Move Can Pot</i>	68%	48%	51%	55%	89%	86%	79%	84%	93%	79%
<i>Move Pillbottle Pad</i>	67%	46%	84%	61%	73%	71%	93%	94%	86%	86%
<i>Move Playingcard Away</i>	74%	65%	96%	84%	93%	98%	96%	99%	94%	90%
<i>Move Stapler Pad</i>	41%	24%	56%	42%	78%	73%	74%	49%	83%	76%
<i>Open Laptop</i>	71%	81%	90%	96%	93%	100%	96%	96%	98%	100%
<i>Open Microwave</i>	4%	32%	34%	77%	79%	71%	91%	75%	83%	72%
<i>Pick Diverse Bottles</i>	69%	31%	81%	71%	58%	36%	79%	86%	58%	75%
<i>Pick Dual Bottles</i>	59%	37%	93%	63%	47%	36%	82%	95%	89%	75%
<i>Place A2B Left</i>	43%	47%	87%	82%	48%	49%	86%	83%	92%	95%
<i>Place A2B Right</i>	39%	34%	87%	84%	36%	36%	74%	77%	90%	87%
<i>Place Bread Basket</i>	62%	46%	77%	64%	81%	71%	92%	93%	83%	84%
<i>Place Bread Skillet</i>	66%	49%	85%	66%	77%	67%	90%	89%	94%	83%
<i>Place Burger Fries</i>	81%	76%	94%	87%	94%	94%	95%	96%	96%	95%
<i>Place Can Basket</i>	55%	46%	62%	62%	49%	52%	68%	78%	87%	77%
<i>Place Cans Plasticbox</i>	63%	45%	94%	84%	97%	98%	97%	100%	89%	92%
<i>Place Container Plate</i>	97%	92%	99%	95%	97%	95%	99%	99%	96%	98%
<i>Place Dual Shoes</i>	59%	51%	75%	75%	79%	88%	80%	83%	96%	94%
<i>Place Empty Cup</i>	91%	85%	100%	99%	100%	98%	100%	100%	99%	99%
<i>Place Fan</i>	66%	71%	87%	85%	80%	75%	91%	79%	94%	91%
<i>Place Mouse Pad</i>	20%	20%	60%	39%	70%	70%	82%	78%	75%	78%
<i>Place Object Basket</i>	67%	70%	80%	76%	44%	39%	90%	91%	84%	66%
<i>Place Object Scale</i>	57%	52%	86%	80%	52%	74%	84%	90%	90%	89%
<i>Place Object Stand</i>	82%	68%	91%	85%	86%	88%	97%	93%	93%	92%
<i>Place Phone Stand</i>	49%	53%	81%	81%	88%	87%	92%	93%	95%	90%
<i>Place Shoe</i>	76%	76%	92%	93%	96%	95%	99%	94%	100%	100%
<i>Press Stapler</i>	44%	37%	87%	83%	92%	98%	90%	88%	96%	97%
<i>Put Bottles Dustbin</i>	65%	56%	84%	79%	74%	77%	88%	92%	88%	85%
<i>Put Object Cabinet</i>	73%	60%	80%	79%	46%	48%	92%	86%	92%	85%
<i>Rotate QRcode</i>	74%	70%	89%	87%	34%	33%	93%	84%	95%	86%
<i>Scan Object</i>	55%	42%	72%	65%	14%	36%	91%	97%	79%	83%
<i>Shake Bottle Horizontally</i>	98%	92%	99%	99%	100%	100%	100%	100%	100%	99%
<i>Shake Bottle</i>	94%	91%	99%	97%	99%	100%	99%	100%	100%	100%
<i>Stack Blocks Three</i>	72%	52%	91%	76%	6%	10%	92%	99%	97%	95%
<i>Stack Blocks Two</i>	93%	79%	97%	100%	92%	87%	100%	100%	100%	98%
<i>Stack Bowls Three</i>	77%	75%	77%	71%	76%	86%	72%	83%	83%	80%
<i>Stack Bowls Two</i>	94%	95%	95%	96%	96%	93%	92%	95%	98%	96%
<i>Stamp Seal</i>	46%	33%	79%	55%	76%	82%	76%	86%	77%	83%
<i>Turn Switch</i>	41%	42%	62%	54%	40%	61%	61%	65%	18%	41%
Average (%)	65.92	58.40	82.74	76.76	72.80	72.84	86.50	85.34	88.42	86.78